

38. upper mesh 収束確認

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	scripts/run_upper_mesh_convergence.py
種別	Python
区分	planning validation
役割	候補 policy や exact replay を距離分解能違いで再計算し、結果が十分収束しているか確認する。
起動文脈	GPU/learned policy の acceptance 前検証。

このファイルを読む理由

候補 policy や exact replay を距離分解能違いで再計算し、結果が十分収束しているか確認する。

- 細かい距離メッシュが本当に必要十分かを調べる。

呼び出し関係

どこから呼ばれるか

- validation pipeline
- 手動検証

次に読むと理解が進むファイル

- scripts/validate_gpu_upper_policy_candidates.py

同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

Run fixed-policy integration and independent-policy control convergence gates.

代表コード断片

```
ROOT = Path(__file__).resolve().parents[1]

def parse_float_list(value: str) -> list[float]:
    return [float(item) for item in str(value).split(",") if item.strip()]

def parse_control_policies(values: list[str]) -> dict[float, Path]:
    policies: dict[float, Path] = {}
    for raw in values:
        spacing_text, separator, path_text = str(raw).partition("=")
        if not separator or not path_text.strip():
            raise ValueError("--control-policy must use CONTROL_KM=POLICY_CSV")
        spacing = float(spacing_text)
        if spacing <= 0.0:
            raise ValueError("control-policy spacing must be positive")
        path = Path(path_text).resolve()
        if not path.is_file():
            raise FileNotFoundError(path)
        policies[spacing] = path
    return policies
```

主要構造の読み取り

主要関数は `parse_float_list`, `parse_control_policies`, `assign_finetest_control_policy`, `file_sha256`, `resolve`, `load_plan`, `speed_rms_difference`, `materialize_profile`。CLI 引数宣言は 7 件。

主要クラス・関数

行	種別	名前	補足
22	function	<code>parse_float_list</code>	args: value
26	function	<code>parse_control_policies</code>	args: values
42	function	<code>assign_finetest_control_policy</code>	args: control_meshes, supplied_policies, finest_policy
57	function	<code>file_sha256</code>	args: path
65	function	<code>resolve</code>	args: profile, value
70	function	<code>load_plan</code>	args: summary
78	function	<code>speed_rms_difference</code>	args: left, right, race_km
85	function	<code>materialize_profile</code>	args: base_path, output_path
131	function	<code>main</code>	

ファイルを上から読んだときの定義順

行	内容
19	ROOT に <code>Path(__file__).resolve().parents[1]</code> の結果を代入する。
22	関数 <code>parse_float_list</code> を定義する。
26	関数 <code>parse_control_policies</code> を定義する。

42 関数 `assign_finetest_control_policy` を定義する。
 57 関数 `file_sha256` を定義する。
 65 関数 `resolve` を定義する。
 70 関数 `load_plan` を定義する。
 78 関数 `speed_rms_difference` を定義する。
 85 関数 `materialize_profile` を定義する。
 131 関数 `main` を定義する。
 308 条件 `__name__ == '__main__'` を判定し、真なら内部処理を行う。

import 群の詳細

行	import 文	このファイルで読む理由
4	<code>from __future_ _import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や <code>forward reference</code> を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
6	<code>import argparse</code>	CLI 引数を宣言し、実行時パラメータを外から受け取るため。このファイル内での主な使用位置は L132, L150。
7	<code>import hashlib</code>	snapshot ID や入力資産の <code>digest</code> を作るため。このファイル内での主な使用位置は L58, L196。
8	<code>import json</code>	<code>manifest</code> 、 <code>checkpoint</code> 、 <code>UDP payload</code> をやり取りするため。このファイル内での主な使用位置は L214, L225, L303, L304。
9	<code>import math</code>	<code>clip</code> 、 <code>sqrt</code> 、 <code>sin/cos</code> 、有限判定など数値ロジックに使うため。このファイル内での主な使用位置は L116。
10	<code>import subprocess</code>	<code>git</code> 情報取得や外部コマンド実行を行うため。このファイル内での主な使用位置は L221。
11	<code>import sys</code>	<code>sys</code> モジュールを利用するため。このファイル内での主な使用位置は L204。
12	<code>from pathlib import Path</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L19, L26, L27, L35, L44, L45, L46, L57, ...。
14	<code>import numpy as np</code>	数値配列、 <code>clip</code> 、補間、統計計算を行うため。このファイル内での主な使用位置は L79, L80, L81, L82。
15	<code>import pandas as pd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L70, L74, L78, L282, L283。
16	<code>import yaml</code>	<code>profile</code> 、 <code>stop</code> 、 <code>schedule</code> 、 <code>summary</code> YAML を読み書きするため。このファイル内での主な使用位置は L86, L127, L159。

関数・クラスを上から順に解説

L22 関数 `parse_float_list`

- 定義: `parse_float_list(value)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `split`, `str`, `strip`

- 戻り値の要点: `[float(item) for item in str(value).split(',') if item.strip()]`

`if item.strip()` を返す。

L26 関数 `parse_control_policies`

- 定義: `parse_control_policies(values)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `FileNotFoundError`, `Path`, `ValueError`, `float`, `is_file`, `partition`, `resolve`, `str`, `strip`
 - 戻り値の要点: `policies`
1. `policies` に `{}` を代入する。
 2. `values` を順に走査し、各要素を `raw` に入れて処理する。
 3. `policies` を返す。

L42 関数 `assign_finetest_control_policy`

- 定義: `assign_finetest_control_policy(control_meshes, supplied_policies, finest_policy)`
 - docstring: Assign -policy to the finest requested control mesh without relabelling it.
 - このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `dict`, `float`, `min`, `resolve`
 - 戻り値の要点: `(finest_ctrl, policies, order)`
1. 条件 `not control_meshes` を判定し、真なら内部処理を行う。
 2. `finest_ctrl` に `min((float(value) for value in control_meshes))` の結果を代入する。
 3. `policies` に `dict(supplied_policies)` の結果を代入する。
 4. `policies[finest_ctrl]` に `finest_policy.resolve()` の結果を代入する。
 5. `order` に `[float(ctrl) for ctrl in control_meshes if float(ctrl) in policies]` の結果を代入する。
 6. `(finest_ctrl, policies, order)` を返す。

L57 関数 `file_sha256`

- 定義: `file_sha256(path)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `hexdigest`, `iter`, `open`, `read`, `sha256`, `update`
 - 戻り値の要点: `digest.hexdigest()`
1. `digest` に `hashlib.sha256()` の結果を代入する。
 2. `with` 文で `path.open('rb')` を管理しながら処理する。
 3. `digest.hexdigest()` を返す。

L65 関数 `resolve`

- 定義: `resolve(profile, value)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `is_absolute`, `resolve`, `str`
- 戻り値の要点: `path if path.is_absolute() else (profile.parent / path).resolve()`

1. `path` に `Path(str(value))` の結果を代入する。
2. `path if path.is_absolute() else (profile.parent / path).resolve()` を返す。

L70 関数 `load_plan`

- 定義: `load_plan(summary)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `is_absolute`, `read_csv`, `resolve`, `sort_values`
- 戻り値の要点: `frame.sort_values('plan_s_km')`

1. `path` に `Path(summary['plan_csv'])` の結果を代入する。
2. 条件 `not path.is_absolute()` を判定し、真なら内部処理を行う。
3. `frame` に `pd.read_csv(path)` の結果を代入する。
4. `frame.sort_values('plan_s_km')` を返す。

L78 関数 `speed_rms_difference`

- 定義: `speed_rms_difference(left, right, race_km)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `arange`, `float`, `interp`, `mean`, `sqrt`
- 戻り値の要点: `float(np.sqrt(np.mean((left_v - right_v) ** 2)))`

1. `grid` に `np.arange(0.0, race_km + 1.0, 1.0)` の結果を代入する。
2. `left_v` に `np.interp(grid, left['plan_s_km'], left['plan_v_kmh'])` の結果を代入する。
3. `right_v` に `np.interp(grid, right['plan_s_km'], right['plan_v_kmh'])` の結果を代入する。
4. `float(np.sqrt(np.mean((left_v - right_v) ** 2)))` を返す。

L85 関数 `materialize_profile`

- 定義: `materialize_profile(base_path, output_path, policy_path, ds_km, ctrl_km)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ceil`, `float`, `get`, `int`, `items`, `list`, `mkdir`, `read_text`, `resolve`, `safe_dump`, `safe_load`, `str`
- 戻り値の要点: `cfg`

1. `cfg` に `yaml.safe_load(base_path.read_text(encoding='utf-8'))` or `{}` の結果を代入する。
2. `list((cfg.get('paths', {}) or {}).items())` を順に走査し、各要素を `(key, value)` に入れて処理する。
3. `('fit_summary_yaml', 'terminal_consistency_yaml')` を順に走査し、各要素を `key` に入れて処理する。
4. `cfg['paths']['initial_upper_policy_csv']` に `str(policy_path.resolve())` の結果を代入する。
5. `race_km` に `float(cfg['mpc']['race_km'])` の結果を代入する。
6. `run_dir` に `output_path.parent` の結果を代入する。
7. `cfg['meta']['name']` に `f'mesh_ds{ds_km:g}_ctrl{ctrl_km:g}'` の結果を代入する。
8. `cfg['simulation'].update(...)` を実行する。
9. `cfg['mpc'].update(...)` を実行する。
10. `output_path.parent.mkdir(...)` を実行する。

L131 関数 main

- 定義: `main()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ArgumentParser`, `DataFrame`, `Path`, `RuntimeError`, `abs`, `add_argument`, `all`, `append`, `assign_finetest_control_policy`, `bool`, `dumps`, `encode`
- 戻り値の要点: 0 if `gate_pass` else 2

1. `parser` に `argparse.ArgumentParser(description=__doc__)` の結果を代入する。
2. `parser.add_argument(...)` を実行する。
3. `parser.add_argument(...)` を実行する。
4. `parser.add_argument(...)` を実行する。
5. `parser.add_argument(...)` を実行する。
6. `parser.add_argument(...)` を実行する。
7. `parser.add_argument(...)` を実行する。
8. `parser.add_argument(...)` を実行する。
9. `args` に `parser.parse_args()` の結果を代入する。
10. `profile` に `args.profile.resolve()` の結果を代入する。

CLI 引数

行	引数
133	<code>--profile</code>
134	<code>--policy</code>
135	<code>--output-dir</code>
136	<code>--integration-meshes-km</code>
137	<code>--control-meshes-km</code>

```
138     --control-policy
148     --resume
```

処理の流れ

1. CLI 引数を解釈し、`main()` から処理を起動する。

このファイルの位置づけ

この資料群では、`scripts/run_upper_mesh_convergence.py` を **planning validation** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。